

Automater 시스템 ERD 해설서

21개 테이블, 각각의 존재 이유와 해결하는 문제

이 문서는 factory/models.py에 정의된 모든 테이블에 대해 세 가지 질문에 답합니다: (1) 왜 이 테이블이 나왔는가, (2) 이 테이블이 없었다면 어떻게 되는가, (3) 이 테이블이 해결하는 문제가 무엇인가.

테이블은 네 개의 Zone으로 분류합니다:

■ Identity — 사용자/계정/플랫폼/미디어

■ Keyword — 키워드 분류/조합/접사

■ Campaign — 캠페인 설정/레이아웃/프리셋

■ Execution — 조합/배치/실행 결과

목차

1. Identity Zone

- 1.1 users
- 1.2 platforms
- 1.3 accounts
- 1.4 media

2. Keyword Zone

- 2.1 keyword_categories
- 2.2 keywords
- 2.3 affixes

3. Campaign Zone

- 3.1 campaigns
- 3.2 campaign_slots
- 3.3 spacing_rules
- 3.4 campaign_keyword_picks
- 3.5 campaign_palettes / palette_items
- 3.6 post_layouts / layout_slots
- 3.7 publish_presets
- 3.8 run_presets

4. Execution Zone

- 4.1 combinations
- 4.2 combination_keywords (association)
- 4.3 batches
- 4.4 batch_items

5. 전체 ERD Zone Map

6. 테이블 간 핵심 질문 Q&A;

1. Identity Zone

시스템에 접근하는 사람(User), 포스팅할 플랫폼(Platform), 플랫폼 위의 블로그 계정(Account), 업로드된 미디어 파일(Media)을 관리합니다.

1.1 users

Identity Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	시스템 내부 식별자
email	VARCHAR(256)	UNIQUE, NOT NULL	로그인 이메일
password_hash	VARCHAR(512)	NOT NULL	bcrypt 해시
display_name	VARCHAR(64)	default ''	UI 표시명
role	VARCHAR(16)	default 'operator'	권한 레벨
status	VARCHAR(16)	default 'active'	계정 상태
last_login_at	DATETIME	NULLABLE	마지막 로그인

WHY: 이 테이블이 나온 이유

이 시스템은 여러 운영자가 각자의 캠페인과 블로그 계정을 관리하는 멀티테넌트 SPA입니다. 운영자마다 자기 소유의 Account, Campaign, Layout, Preset, Media를 가집니다. 이 소유권의 루트가 users 테이블입니다.

WITHOUT: 이 테이블이 없다면

계정 소유권을 구분할 수 없습니다. 모든 운영자가 모든 블로그 계정과 캠페인을 볼 수 있고, 누가 어떤 캠페인을 만들었는지 추적할 수 없습니다. role 기반 권한 분리도 불가능해져 운영자가 관리자 기능에 접근할 수 있습니다.

SOLVES: 이 테이블이 해결하는 문제

멀티테넌트 격리. 운영자 A의 캠페인/계정/미디어는 운영자 B에게 보이지 않습니다. 모든 하위 테이블이 user_id FK로 소유권을 추적하며, API 레이어에서 현재 세션의 user_id로 필터링합니다.

EXAMPLE: 실제 동작 예시

운영자 김철수(user_id=1)가 네이버 계정 3개, 캠페인 2개, 이미지 50개를 소유합니다. 운영자 이영희(user_id=2)도 자기만의 계정과 캠페인을 가집니다. API에서 GET /accounts를 호출하면 현재 세션의 user_id로 필터링됩니다: SELECT * FROM accounts WHERE user_id = 1. 김철수는 이영희의 계정 목록을 절대 볼 수 없습니다. role='admin'인 사용자만 모든 운영자의 데이터를 볼 수 있습니다.

1.2 platforms

Identity Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
name	VARCHAR(64)	NOT NULL	표시명 (예: 네이버 블로그)
slug	VARCHAR(32)	UNIQUE	코드 식별자 (예: naver)
base_url	VARCHAR(256)	default ''	플랫폼 기본 URL
status	VARCHAR(16)	default 'active'	활성 상태

WHY: 이 테이블이 나온 이유

시스템은 네이버 블로그 외에 WordPress, Tistory 등 다양한 플랫폼을 지원할 수 있도록 설계되었습니다. 각 플랫폼은 고유한 로그인 방식, 에디터, URL 구조를 가집니다. 이 차이를 하드코딩하지 않고 데이터로 관리하기 위해 platforms 테이블이

존재합니다.

► **WITHOUT: 이 테이블이 없다면**

플랫폼이 코드에 하드코딩됩니다. 새 플랫폼을 추가할 때마다 코드 수정과 배포가 필요합니다. Account가 어떤 플랫폼의 계정인지 구분할 수 없어, 네이버 계정으로 WordPress에 글을 올리려는 오류를 방지할 수 없습니다.

► **SOLVES: 이 테이블이 해결하는 문제**

플랫폼 확장성. DB에 행 하나를 추가하는 것으로 새 플랫폼을 등록할 수 있습니다. Account.platform_id와 Campaign.platform_id가 이 테이블을 참조하여 계정-캠페인-플랫폼 간 정합성을 보장합니다.

► **EXAMPLE: 실제 동작 예시**

현재 시스템에는 Platform(id=1, slug='naver_blog', base_url='https://blog.naver.com')이 등록되어 있습니다. 나중에 WordPress를 추가하려면 INSERT INTO platforms (name, slug, base_url) VALUES ('WordPress', 'wordpress', 'https://example.wordpress.com')만 실행하면 됩니다. Account(platform_id=1)은 네이버 계정, Account(platform_id=2)는 WordPress 계정이 됩니다. Campaign(platform_id=1)에 Account(platform_id=2)를 연결하려 하면 비즈니스 로직에서 플랫폼 불일치를 감지할 수 있습니다.

1.3 accounts

Identity Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
user_id	INTEGER FK	users.id	소유 운영자
platform_id	INTEGER FK	platforms.id	플랫폼
username	VARCHAR(128)	NOT NULL	블로그 로그인 ID
password_enc	VARCHAR(512)	default ''	암호화된 비밀번호
extra	JSON	NULLABLE	플랫폼별 추가 정보 (blog_id 등)
last_used_at	DATETIME	NULLABLE	마지막 포스팅 시각
cooldown_days	INTEGER	default 14	재사용 대기 일수
status	VARCHAR(16)	default 'active'	active/cooling/banned

► **WHY: 이 테이블이 나온 이유**

한 운영자가 여러 블로그 계정을 운용합니다. 각 계정은 특정 플랫폼에 속하고, 독립적인 로그인 정보와 사용 이력을 가집니다. 특히 cooldown_days는 계정 감지를 피하기 위한 핵심 비즈니스 규칙입니다.

► **WITHOUT: 이 테이블이 없다면**

계정 로테이션이 불가능합니다. 하나의 계정으로 모든 포스팅을 하면 플랫폼의 스팸 탐지에 걸릴 가능성이 급격히 높아집니다. cooldown 관리가 없으면 BatchDispatcher가 '쉬어야 할' 계정에 작업을 할당합니다.

► **SOLVES: 이 테이블이 해결하는 문제**

계정 풀 관리와 쿨다운 로테이션. BatchDispatcher._available_accounts()는 last_used_at + cooldown_days를 현재 시각과 비교하여 사용 가능한 계정만 반환합니다. UNIQUE(platform_id, username)은 같은 플랫폼에 동일 ID가 중복 등록되는 것을 방지합니다.

► **EXAMPLE: 실제 동작 예시**

운영자가 네이버 계정 3개를 등록했습니다: Account(username='blog_a', cooldown_days=14, last_used_at=3월 1일, status='cooling'), Account(username='blog_b', cooldown_days=14, last_used_at=None, status='active'), Account(username='blog_c', cooldown_days=7, last_used_at=3월 10일, status='active'). 오늘이 3월 20일이면 BatchDispatcher._available_accounts()는 다음을 판정합니다: blog_a → status='cooling'이므로 제외. blog_b →

last_used_at=None(한 번도 안 씀)이므로 최우선 선택. blog_c → 3월 10일 + 7일 = 3월 17일 < 3월 20일이므로 쿨다운 완료, 선택 가능. 결과적으로 [blog_b, blog_c] 순서로 반환됩니다 (미사용 계정 우선, 그 다음 last_used_at 오름차순). 배분 후 blog_b.status는 'cooling'으로, blog_b.last_used_at은 현재 시각으로 갱신됩니다.

extra JSON은 플랫폼별로 다른 추가 정보를 담습니다. 예: 네이버의 blog_id, WordPress의 api_url.

1.4 media

Identity Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	media_id (LayoutSlot.config에서 참조)
user_id	INTEGER FK	users.id	소유 운영자
original_name	VARCHAR(256)	NOT NULL	업로드 시 원본 파일명
content_type	VARCHAR(64)	NOT NULL	MIME type
storage_path	VARCHAR(512)	UNIQUE	디스크 상대 경로
size_bytes	INTEGER	NOT NULL	파일 크기

► WHY: 이 테이블이 나온 이유

포스트에 삽입되는 이미지는 운영자가 미리 업로드합니다. LayoutSlot의 config JSON에서 media_id로 참조되며, media_resolver가 이 테이블을 조회하여 실제 파일 경로를 반환합니다.

► WITHOUT: 이 테이블이 없다면

이미지 경로를 LayoutSlot.config에 절대 경로로 직접 기록해야 합니다. 파일이 이동하거나 스토리지 구조가 바뀌면 모든 Layout을 수정해야 합니다. 또한 사용자별 미디어 격리가 불가능하여 다른 운영자의 이미지를 참조할 수 있습니다.

► SOLVES: 이 테이블이 해결하는 문제

미디어 간접 참조와 소유권 격리. LayoutSlot은 media_id(정수)만 저장하고, 런타임에 media_resolver가 media.storage_path를 통해 절대 경로를 해석합니다. 스토리지 이전 시 media.storage_path만 업데이트하면 모든 Layout이 자동으로 따라갑니다.

► EXAMPLE: 실제 동작 예시

운영자가 '강남_과외_ 쌤네일.jpg'를 업로드합니다. 시스템이 저장: Media(id=42, user_id=1, original_name='강남_과외_ 쌤네일.jpg', storage_path='1/a3f2e8c9d1.jpg', size_bytes=245000). 이제 LayoutSlot에서 이 이미지를 참조할 때: LayoutSlot(block_type='featured', config={"media_id": 42, "overlay_text": "{keyword}"}). 런타임에 media_resolver(42)가 호출되면: DB에서 Media(id=42)를 조회 → storage_path='1/a3f2e8c9d1.jpg' → LocalStorage.resolve('1/a3f2e8c9d1.jpg') → '/app/uploads/1/a3f2e8c9d1.jpg' 반환. 나중에 uploads 디렉토리를 S3로 이전하면, LocalStorage를 S3Storage로 교체하기만 하면 됩니다. LayoutSlot의 config는 여전히 {"media_id": 42}로 동일합니다.

2. Keyword Zone

SEO 키워드의 분류 체계(Category), 개별 키워드(Keyword), 그리고 키워드 파생 규칙(Affix)을 관리합니다. 이 세 테이블은 ComboGenerator의 카르테시안 곱의 입력 재료입니다.

2.1 keyword_categories

Keyword Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
name	VARCHAR(64)	NOT NULL	표시명 (예: 지역)
slug	VARCHAR(32)	UNIQUE	코드 식별자 (예: region)

► WHY: 이 테이블이 나온 이유

키워드는 '강남', '수학', '1:1' 같은 단어이지만, 이것들은 각각 '지역(region)', '과목(subject)', '수업유형(learning_type)' 같은 분류에 속합니다. 카르테시안 곱을 만들려면 '어떤 축의 키워드인지'를 알아야 합니다.

► WITHOUT: 이 테이블이 없다면

모든 키워드가 평평한 하나의 목록이 됩니다. '강남'과 '수학'을 어떻게 곱해야 하는지, '{region} {subject}'에서 어떤 키워드가 region에 들어가는지 알 수 없습니다. TitleOption.template의 slug 보간이 완전히 동작하지 않습니다.

► SOLVES: 이 테이블이 해결하는 문제

키워드의 차원(axis) 정의. category.slug가 TitleOption.template의 {region}, {subject}와 1:1 대응합니다. CampaignSlot이 '이 캠페인은 region, subject 두 축을 사용한다'를 선언하면, ComboGenerator가 각 축의 키워드를 카르테시안 곱합니다.

► EXAMPLE: 실제 동작 예시

실제 테스트 데이터에는 3개의 카테고리가 존재합니다: KeywordCategory(slug='region', name='지역'), KeywordCategory(slug='subject', name='과목'), KeywordCategory(slug='learning_type', name='학습형태'). Campaign.title_template이 '{region} {subject} {learning_type}'이면, job_builder._build_values(combo)가 combo.keywords를 순회하며 {region: '강남', subject: '수학', learning_type: '과외'} dict를 만듭니다. 이 dict의 키가 바로 category.slug이고, template의 {region} 등과 매칭됩니다. 만약 카테고리 테이블이 없으면 '강남'이 region인지 subject인지 알 방법이 없습니다.

2.2 keywords

Keyword Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
category_id	INTEGER FK	keyword_categories.id	소속 카테고리
parent_id	INTEGER FK	keywords.id, NULLABLE	상위 키워드 (계층)
value	VARCHAR(128)	NOT NULL	실제 키워드 값 (예: 강남구)
display_value	VARCHAR(128)	default ''	UI 표시 값
tier	INTEGER	default 1	우선순위 티어
active	BOOLEAN	default True	활성 여부
sort_order	INTEGER	default 0	정렬 순서
metadata	JSON	NULLABLE	추가 메타데이터

▶ WHY: 이 테이블이 나온 이유

개별 SEO 키워드를 저장합니다. 각 키워드는 하나의 카테고리에 속하며, parent_id로 계층 구조를 표현할 수 있습니다 (예: 서울 > 강남구). active 플래그로 일시 비활성화가 가능합니다.

▶ WITHOUT: 이 테이블이 없다면

키워드를 코드나 설정 파일에 하드코딩해야 합니다. 새 지역이나 과목이 추가될 때마다 코드 배포가 필요합니다. 키워드별 활성/비활성 전환이나 우선순위 관리가 불가능합니다.

▶ SOLVES: 이 테이블이 해결하는 문제

키워드의 동적 관리와 필터링. ComboGenerator는 active=True인 키워드만 조합에 포함합니다. UNIQUE(category_id, value)는 같은 카테고리에 동일 키워드 중복을 방지합니다. tier와 sort_order로 우선순위 기반 선별이 가능합니다.

▶ EXAMPLE: 실제 동작 예시

region 카테고리에 3개의 키워드가 등록됩니다: Keyword(value='강남구', active=True, sort_order=1), Keyword(value='수원시', active=True, sort_order=2), Keyword(value='성남시', active=False, sort_order=3). ComboGenerator._build_keyword_groups()는 WHERE active = True를 적용하므로 실제 조합에는 [강남구, 수원시]만 포함됩니다. 성남시 캠페인을 일시 중단하고 싶으면 active=False로 토글하면 끝입니다. 삭제하지 않으므로 나중에 다시 active=True로 복원할 수 있습니다. 테스트 코드에서 확인: test_inactive_keyword_excluded()가 성남시가 조합에 포함되지 않음을 검증합니다.

2.3 affixes

■ Keyword Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
keyword_id	INTEGER FK	keywords.id, CASCADE	대상 키워드
type	VARCHAR(16)	NOT NULL	'prefix' 또는 'suffix'
value	VARCHAR(64)	NOT NULL	접사 값 (예: '구', '동')
sort_order	INTEGER	default 0	적용 순서
active	BOOLEAN	default True	활성 여부

▶ WHY: 이 테이블이 나온 이유

키워드 '강남구'를 제목에 사용할 때, 어떤 캠페인에서는 '강남'으로(접미사 '구' 제거), 다른 캠페인에서는 '강남구' 그대로 사용하고 싶습니다. 이 접사 규칙을 키워드 자체를 복제하지 않고 관리하기 위한 테이블입니다.

▶ WITHOUT: 이 테이블이 없다면

'강남구'와 '강남'을 별도 키워드로 등록해야 합니다. 같은 지역인데 표기만 다른 키워드가 무한히 증식합니다. 키워드 수가 N이면 표기 변형까지 합치면 N x M이 되어 관리가 불가능합니다.

▶ SOLVES: 이 테이블이 해결하는 문제

키워드 파생 규칙의 데이터화. job_builder._apply_affixes()가 active=False인 접사를 자동 제거합니다. 예: Keyword('강남구') + Affix(suffix='구', active=False) = '강남'. active 토글만으로 같은 키워드의 표기를 캠페인별로 제어할 수 있습니다.

▶ EXAMPLE: 실제 동작 예시

테스트 픽스처에서 Keyword(value='강남구')에 세 개의 Affix가 등록됩니다: Affix(type='suffix', value='구', sort_order=0, active=True), Affix(type='suffix', value='동', sort_order=1, active=True), Affix(type='prefix', value='동', sort_order=0, active=True). 모두 active=True이므로 _apply_affixes('강남구', affixes)는 '강남구' 그대로 반환합니다 (active인 접사는 '의도적으로 유지'를 의미). 이제 suffix '구'를 active=False로 토글하면: _apply_affixes('강남구',

affixes)는 '구'를 제거하여 '강남'을 반환합니다. 제목 템플릿 '{region} {subject} 과외'에서 '강남구 수학 과외' 대신 '강남 수학 과외'가 됩니다. 테스트 코드 확인: `test_job_builder.py`의 `TestBuildValues.test_inactive_suffix_stripped()`.

3. Campaign Zone

캠페인은 '무엇을, 어떤 형식으로, 언제, 어떻게 포스팅할 것인가'의 모든 설정을 담은 컨테이너입니다. 레이아웃, 발행 프리셋, 실행 프리셋 세 가지를 선택적으로 조합합니다.

3.1 campaigns

■ Campaign Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
user_id	INTEGER FK	users.id	소유자
platform_id	INTEGER FK	platforms.id	대상 플랫폼
layout_id	INTEGER FK	post_layouts.id, NULLABLE	포스트 레이아웃
publish_preset_id	INTEGER FK	publish_presets.id, NULLABLE	발행 설정
run_preset_id	INTEGER FK	run_presets.id, NULLABLE	실행 설정
title_template	VARCHAR(256)	default ''	제목 템플릿 (예: {region}{subject})
config	JSON	NULLABLE	캠페인별 추가 설정
status	VARCHAR(16)	default 'active'	활성 상태

► WHY: 이 테이블이 나온 이유

하나의 캠페인은 '강남 지역 수학 과외 홍보'처럼 특정 목적의 포스팅 프로젝트입니다. 어떤 플랫폼에, 어떤 레이아웃으로, 어떤 발행/실행 규칙으로 포스팅할지를 세 개의 nullable FK로 조합합니다.

► WITHOUT: 이 테이블이 없다면

레이아웃, 발행 옵션, 실행 옵션이 캠페인마다 개별적으로 하드코딩됩니다. 같은 레이아웃을 10개 캠페인에서 쓰고 싶으면 10번 복사해야 합니다. 프리셋 변경 시 모든 캠페인을 일일이 수정해야 합니다.

► SOLVES: 이 테이블이 해결하는 문제

3-way 선택적 조합. layout_id, publish_preset_id, run_preset_id가 모두 nullable이므로 설정하지 않으면 시스템 기본값을 사용합니다. 하나의 레이아웃을 여러 캠페인이 공유할 수 있고, 프리셋 변경이 참조하는 모든 캠페인에 즉시 반영됩니다.

► EXAMPLE: 실제 동작 예시

테스트에서 세 가지 설정 조합을 검증합니다: (1) Campaign(layout_id=None, publish_preset_id=None, run_preset_id=None) — 모든 설정이 null이면 job_builder가 기본 3-ParagraphBlock body, preset_loader가 기본 PublishOption(mode='fixed'), 기본 RunSetting을 생성합니다. (2) Campaign(layout_id=5, publish_preset_id=3, run_preset_id=2) — 세 가지 모두 지정하면 해당 프리셋에서 설정을 로드합니다. (3) PublishPreset(id=3, config={"mode": "immediate"})를 공유하는 캠페인 2개 — test_multiple_campaigns_share_preset()이 검증하듯, pub.campaigns의 길이가 2이고, 프리셋 수정 시 양쪽 캠페인에 동시 반영됩니다. Campaign에서 프리셋을 분리하려면 run_preset_id=None으로 설정하면 끝입니다 (test_detach_preset_sets_null).

3.2 campaign_slots

■ Campaign Zone

Column	Type	Constraint	Description
campaign_id	INTEGER FK	PK, campaigns.id	캠페인
category_id	INTEGER FK	PK, keyword_categories.id	키워드 카테고리
sort_order	INTEGER	default 0	카테고리 순서

▶ WHY: 이 테이블이 나온 이유

캠페인마다 사용하는 키워드 축(카테고리)이 다릅니다. 캠페인 A는 [region, subject]만 사용하고, 캠페인 B는 [region, subject, learning_type]을 사용합니다. 이 '이 캠페인은 어떤 카테고리를 사용하는가'를 선언합니다.

▶ WITHOUT: 이 테이블이 없다면

모든 캠페인이 모든 카테고리를 사용한다고 가정해야 합니다. 카테고리가 5개이면 카르테시안 곱이 5차원이 되어 조합 수가 폭발합니다. 캠페인별로 필요한 축만 선택하는 유연성이 없습니다.

▶ SOLVES: 이 테이블이 해결하는 문제

캠페인별 키워드 차원 선택. ComboGenerator._load_slots()가 이 테이블을 조회하여 '이 캠페인의 축 목록'을 결정합니다. sort_order로 {region} {subject} 순서를 제어하여 제목 생성 순서를 결정합니다.

▶ EXAMPLE: 실제 동작 예시

테스트 픽스처에서 하나의 캠페인에 3개의 slot이 등록됩니다: CampaignSlot(campaign_id=1, category_id=region_cat.id, sort_order=0), CampaignSlot(campaign_id=1, category_id=subject_cat.id, sort_order=1), CampaignSlot(campaign_id=1, category_id=lt_cat.id, sort_order=2). 이 캠페인에서 active 키워드가 region=2, subject=2, learning_type=2이면 ComboGenerator는 $2 \times 2 \times 2 = 8$ 개 조합을 생성합니다 (test_inserted_count_matches_cartesian_product에서 검증). 만약 다른 캠페인이 slot을 [region, subject]만 가지면 $2 \times 2 = 4$ 개만 생성됩니다. sort_order=0인 region이 title_template '{region} {subject}'에서 첫 번째 위치에 대응합니다.

3.3 spacing_rules

■ Campaign Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
campaign_id	INTEGER FK	campaigns.id	캠페인
pattern	JSON	NOT NULL	조합 변형 패턴
active	BOOLEAN	default True	활성 여부

▶ WHY: 이 테이블이 나온 이유

같은 키워드 조합이라도 '강남 수학'과 '강남수학'(띄어쓰기 변형) 같은 표기 변형을 만들어야 합니다. SpacingRule은 이 변형 규칙을 정의하며, 카르테시안 곱의 추가 차원으로 작용합니다.

▶ WITHOUT: 이 테이블이 없다면

키워드 조합의 띄어쓰기/표기 변형을 수동으로 관리해야 합니다. region=2, subject=2이면 4개 조합인데, 변형이 3가지면 수동으로 12개를 만들어야 합니다.

▶ SOLVES: 이 테이블이 해결하는 문제

조합 변형의 자동화. build_combos(kw_groups, spacing_rule_ids)에서 spacing_rule_ids가 추가 차원으로 곱해집니다. 키워드 N개 x 규칙 M개 = N*M개 Combination이 자동 생성됩니다.

▶ EXAMPLE: 실제 동작 예시

테스트 픽스처에서 SpacingRule(pattern={"region": 1, "subject": 1, "learning_type": 0})이 등록됩니다. pattern의 값은 해당 카테고리 키워드 사이에 넣을 공백 수를 의미합니다 (0이면 붙여쓰기). 이 하나의 규칙으로 '강남 수학 과외'가 생성됩니다 (region과 subject 사이 1칸, learning_type 앞 0칸). 만약 SpacingRule을 하나 더 추가하면(예: 모두 0칸 → '강남수학과외'), build_combos()의 순수 함수 테스트에서 확인하듯 build_combos([[1,2], [3,4], [5,6]], [10, 20, 30])은 $2 \times 2 \times 2 \times 3 = 24$ 개를 반환합니다. 규칙이 모두 active=False이면 ComboGenerator.run()은 0을 반환합니다 (test_inactive_spacing_rule_returns_zero).

3.4 campaign_keyword_picks

■ Campaign Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
campaign_id	INTEGER FK	campaigns.id	캠페인
category_id	INTEGER FK	keyword_categories.id	카테고리
keyword_id	INTEGER FK	keywords.id	선택된 키워드

▶ WHY: 이 테이블이 나온 이유

region 카테고리에 100개의 키워드가 있지만, 특정 캠페인에서는 '강남구, 서초구' 2개만 사용하고 싶을 수 있습니다. 이 '캠페인별 키워드 필터'를 저장합니다.

▶ WITHOUT: 이 테이블이 없다면

전체 키워드를 쓰거나, 쓰지 않거나 양자택일입니다. 100개 지역 중 2개만 골라 캠페인을 돌리는 것이 불가능합니다. 키워드를 비활성화하면 다른 캠페인에서도 사라집니다.

▶ SOLVES: 이 테이블이 해결하는 문제

캠페인별 키워드 서브셋 선택. CampaignKeywordPicker.load_pick_ids()가 CategoryPicks(dict[int, list[int]|None])를 반환합니다. None이면 '모든 active 키워드', list[int]면 '선택된 ID만' 사용합니다. 키워드 활성/비활성과 독립적으로 캠페인 단위 필터가 동작합니다.

▶ EXAMPLE: 실제 동작 예시

region에 [강남구, 수원시, 성남시(inactive)]가 있고, subject에 [수학, 영어]가 있습니다. 운영자가 이 캠페인에서 region을 '강남구'만 사용하고 싶다면: picker.save_picks(category_slug='region', values=['강남구']). load_pick_ids() 결과: {region_cat_id: [강남구_id], subject_cat_id: None, lt_cat_id: None}. None인 subject와 learning_type은 모든 active 키워드를 사용합니다. ComboGenerator에 이 picks를 전달하면: region 1개 x subject 2개 x lt 2개 x spacing 1개 = 4개 조합 (원래 8개에서 절반). test_picks_filter_reduces_count()가 이 동작을 검증합니다. 나중에 save_picks('region', [])를 호출하면 picks가 제거되어 다시 전체 키워드를 사용합니다 (test_empty_values_clears_picks). 주의: inactive 키워드 '성남시'는 pick 대상에서도 제외됩니다 (test_inactive_keyword_raises).

3.5 campaign_palettes / palette_items

■ Campaign Zone

Column	Type	Constraint	Description
palettes.id	INTEGER PK		
palettes.campaign_id	INTEGER FK	campaigns.id	캠페인
palettes.slug	VARCHAR(64)	UNIQUE(campaign, slug)	폴 이름 (salt_prefix, salt_suffix, cta)
palettes.strategy	VARCHAR(16)	default 'random'	random 또는 sequential
items.palette_id	INTEGER FK	palettes.id, CASCADE	소속 palette
items.value	VARCHAR(256)	NOT NULL	개별 값 (예: '강력 추천')
items.active	BOOLEAN	default True	활성 여부

▶ WHY: 이 테이블이 나온 이유

제목 생성 시 '{salt}' 토큰에 들어갈 접두/접미 문구 폴이 필요합니다. 예전에는 salts.json 파일이었으나, 캠페인마다 다른 폴을 쓰고 싶고, UI에서 동적으로 편집하고 싶어 DB로 이전했습니다.

▶ WITHOUT: 이 테이블이 없다면

salt 값을 JSON 파일이나 코드에 하드코딩해야 합니다. 캠페인마다 다른 salt 풀을 쓰려면 파일을 복사해야 하고, UI에서 실시간 편집이 불가능합니다.

► SOLVES: 이 테이블이 해결하는 문제

캠페인별 런타임 샘플링 풀. `title_generator.generate_title()`에서 `TitleOption.prefix_salts/suffix_salts`로 전달되며, `random/sequential` 전략으로 값을 선택합니다. `UNIQUE(campaign_id, slug)`로 한 캠페인에 같은 slug의 palette가 중복되지 않습니다.

► EXAMPLE: 실제 동작 예시

테스트 픽스처에서 캠페인에 2개의 palette가 등록됩니다: `CampaignPalette(slug='salt_prefix', strategy='random')` → `PalettItem(value='검증된')`, `PalettItem(value='전문')`. `CampaignPalette(slug='salt_suffix', strategy='random')` → `PalettItem(value='강력 추천')`, `PalettItem(value='즉시 가능')`. `Campaign.title_template`이 '{salt} {region} {subject} 과외 {salt}'이면, `TitleOption`이 `prefix_salts=('검증된', '전문')`, `suffix_salts=('강력 추천', '즉시 가능')`으로 생성됩니다. `title_generator._pick_salt_pool()`은 {salt} 토큰의 위치를 봅니다: 첫 번째 {salt}는 `idx=0`(접두 위치)이므로 `prefix_salts`에서, 마지막 {salt}는 `idx=last`이므로 `suffix_salts`에서 랜덤 선택합니다. 결과 예시: '전문 강남 수학 과외 즉시 가능'. 같은 캠페인에서 다음 조합은 '검증된 수원 영어 과외 강력 추천'이 될 수 있습니다. 특정 salt를 일시 중지하려면 `PalettItem.active=False`로 토크합니다.

3.6 post_layouts / layout_slots

■ Campaign Zone

Column	Type	Constraint	Description
layouts.id	INTEGER PK		
layouts.user_id	INTEGER FK	users.id	소유자
layouts.name	VARCHAR(128)	NOT NULL	레이아웃 이름
slots.layout_id	INTEGER FK	post_layouts.id, CASCADE	소속 레이아웃
slots.sort_order	INTEGER	NOT NULL	블록 순서
slots.block_type	VARCHAR(32)	NOT NULL	paragraph/image/heading/...
slots.config	JSON	NULLABLE	블록 설정 ({keyword}, media_id 등)

► WHY: 이 테이블이 나온 이유

포스트의 구조(제목 > 본문 > 이미지 > 본문 > CTA)를 재사용 가능한 템플릿으로 정의합니다. 하나의 레이아웃을 여러 캠페인이 공유할 수 있고, `LayoutSlot`의 `config` JSON에 {keyword} 같은 placeholder를 넣어 키워드에 따라 동적으로 내용이 채워집니다.

► WITHOUT: 이 테이블이 없다면

포스트 구조를 캠페인마다 코드로 정의해야 합니다. '제목 후 본문 3개, 이미지 1개' 같은 구조를 바꾸려면 코드 수정이 필요합니다. 비개발자가 포스트 구조를 변경할 수 없습니다.

► SOLVES: 이 테이블이 해결하는 문제

포스트 구조의 데이터 구동 (data-driven) 설계. `job_builder._build_body_from_layout()`이 slots를 순회하며 `block_factory.create_block()`으로 각 slot을 Block dataclass로 변환합니다. UI에서 드래그앤드롭으로 블록 순서를 바꾸면 `sort_order`만 변경됩니다.

► EXAMPLE: 실제 동작 예시

테스트 픽스처(`test_preset_models.py`)에서 'edu-3-para' 레이아웃이 생성됩니다: `LayoutSlot(sort_order=0, block_type='heading', config={"level": 2, "text": "{keyword} guide"})`, `LayoutSlot(sort_order=1, block_type='paragraph', config={"keyword": "{keyword}", "tone": "review"})`, `LayoutSlot(sort_order=2, block_type='paragraph', config={"keyword": "{keyword}", "tone": "promotional"})`.

Combination(keywords=[강남, 수학])과 이 레이아웃을 결합하면: slot 0 → block_factory.create_block('heading', {'level':2, "text": "{keyword} guide"}, keyword='강남 수학') → HeadingBlock(level=2, text='강남 수학 guide'). slot 1 → ParagraphBlock(keyword='강남 수학', tone='review'). slot 2 → ParagraphBlock(keyword='강남 수학', tone='promotional'). 같은 레이아웃으로 (수원, 영어) 조합을 돌리면: HeadingBlock(text='수원 영어 guide'), ParagraphBlock(keyword='수원 영어', tone='review'), ... 레이아웃 구조는 동일하되 내용만 키워드에 따라 달라집니다. 이미지 slot에 media_id를 넣을 수도 있습니다: LayoutSlot(block_type='featured', config={"media_id": 42, "overlay_text": "{keyword}"}) → media_resolver(42) → FeaturedImageBlock(path='/uploads/1/a3f2e8c9d1.jpg', overlay_text='강남 수학').

3.7 publish_presets

■ Campaign Zone

Column	Type	Constraint	Description
id	INTEGER PK		
user_id	INTEGER FK	users.id	소유자
name	VARCHAR(128)	NOT NULL	프리셋 이름
config	JSON	NOT NULL	PublishOption 필드를 JSON으로 저장

► WHY: 이 테이블이 나온 이유

PublishOption(mode, jitter_minutes, tags, visibility 등)을 재사용 가능한 프리셋으로 관리합니다. '즉시 발행', '30분 랜덤 예약', '비공개 발행' 같은 설정을 미리 만들어두고 캠페인에 연결합니다.

► WITHOUT: 이 테이블이 없다면

발행 옵션을 캠페인마다 개별 컬럼으로 저장해야 합니다. 10개 캠페인이 같은 발행 규칙을 쓸 때 10번 중복 저장됩니다. 규칙 변경 시 10개를 모두 수정해야 합니다.

► SOLVES: 이 테이블이 해결하는 문제

발행 규칙의 재사용과 중앙 관리. preset_loader.load_publish(config, scheduled_at)가 JSON config를 PublishOption frozen dataclass로 역직렬화합니다. 프리셋 하나를 수정하면 참조하는 모든 캠페인에 즉시 반영됩니다.

► EXAMPLE: 실제 동작 예시

test_preset_models.py에서 PublishPreset(name='edu-publish', config={"mode": "random_window", "jitter_minutes": 45, "min_tags": 15})이 생성됩니다. 이 프리셋을 참조하는 캠페인에서 build_posting_spec()이 호출되면: pub_preset.config = {"mode": "random_window", "jitter_minutes": 45, "min_tags": 15}. preset_loader.load_publish(config, scheduled_at=3월 20일 10:00)이 실행되면: config에 at=3월 20일 10:00을 주입한 후 PublishOption(**filtered_config)를 생성합니다. 결과: PublishOption(mode='random_window', at=3월 20일 10:00, jitter_minutes=45, min_tags=15). ContentBuilder.resolve_schedule()은 이 옵션을 보고 10:00 ± 45분(=9:15~10:45) 범위에서 무작위 시각을 선택합니다. 프리셋을 안 쓰는 캠페인(publish_preset_id=None)은 load_publish(None, scheduled_at) → PublishOption(mode='fixed', at=scheduled_at) 기본값을 받습니다.

3.8 run_presets

■ Campaign Zone

Column	Type	Constraint	Description
id	INTEGER PK		
user_id	INTEGER FK	users.id	소유자
name	VARCHAR(128)	NOT NULL	프리셋 이름
config	JSON	NOT NULL	RunSetting 필드를 JSON으로 저장

▶ WHY: 이 테이블이 나온 이유

RunSetting(post_interval, max_daily_posts, headless, upload_delay_ms 등)을 재사용 가능한 프리셋으로 관리합니다. '빠른 실행(headless, 30초 간격)', '안전 실행(GUI, 120초 간격)' 같은 프로파일입니다.

▶ WITHOUT: 이 테이블이 없다면

publish_presets와 같은 문제. 실행 옵션 중복 저장, 일괄 변경 불가.

▶ SOLVES: 이 테이블이 해결하는 문제

실행 프로파일의 재사용. preset_loader.load_setting(config)가 RunSetting으로 역직렬화합니다. publish_presets와 동일한 패턴으로, 캠페인의 run_preset_id가 null이면 시스템 기본값을 사용합니다.

▶ EXAMPLE: 실제 동작 예시

test_preset_models.py에서 RunPreset(name='safe-mode', config={"post_interval": 120, "headless": True, "max_daily_posts": 5})이 생성됩니다. 이 프리셋을 참조하는 캠페인에서 build_posting_spec()이 호출되면: load_setting(config) → RunSetting(post_interval=120, headless=True, max_daily_posts=5, upload_delay_ms=1500(기본값)). config에 없는 필드(upload_delay_ms)는 dataclass 기본값으로 채워집니다. config에 미래에 추가될 필드(예: 'retry_count')가 있어도 _deserialize()가 RunSetting에 없는 필드를 무시하므로 하위 호환이 유지됩니다. 실전 활용: 'fast-mode'(post_interval=30, headless=True)와 'debug-mode'(post_interval=300, headless=False, slow_mo=100)를 만들어 두고 캠페인의 run_preset_id만 바꿔 실행 전략을 전환합니다.

4. Execution Zone

키워드 조합(Combination), 조합을 배치에 묶는 과정(Batch/BatchItem), 그리고 조합과 키워드를 연결하는 다대다 관계(combination_keywords)를 관리합니다. 이 Zone이 Factory Layer의 런타임 데이터입니다.

4.1 combinations

■ Execution Zone

Column	Type	Constraint	Description
id	BIGINT PK	AUTO	
campaign_id	INTEGER FK	campaigns.id	소속 캠페인
spacing_rule_id	INTEGER FK	spacing_rules.id, NULLABLE	적용된 규칙
config	JSON	NULLABLE	추가 설정
used_at	DATETIME	NULLABLE	배치에 배분된 시각

▶ WHY: 이 테이블이 나온 이유

campaign_slots의 카테고리별 키워드를 카르테시안 곱한 결과 한 행이 하나의 Combination입니다. Combination 하나가 포스트 하나에 대응합니다. used_at이 NULL이면 아직 배분되지 않은 '대기 중' 조합입니다.

▶ WITHOUT: 이 테이블이 없다면

어떤 키워드 조합이 이미 포스팅되었는지 추적할 수 없습니다. 같은 조합으로 중복 포스팅이 발생하거나, 아직 처리되지 않은 조합을 식별할 수 없습니다.

▶ SOLVES: 이 테이블이 해결하는 문제

조합의 생명주기 관리. used_at IS NULL → BatchDispatcher._pending_combinations()에서 대기 조합으로 조회. 배분 시 used_at이 채워져 '소비됨'으로 전환. BIGINT PK는 대규모 캠페인(수만 개 조합)을 수용합니다.

▶ EXAMPLE: 실제 동작 예시

캠페인에 region=[강남구, 수원시], subject=[수학, 영어], lt=[과외, 학원], spacing_rule 1개가 있으면 ComboGenerator.run()이 $2 \times 2 \times 2 \times 1 = 8$ 개의 Combination 행을 INSERT합니다. 모두 used_at=NULL 상태로 생성됩니다. BatchDispatcher.dispatch()가 호출되면: _pending_combinations() → WHERE used_at IS NULL ORDER BY id → 8개 반환. 가용 Account(blog_b)에 8개를 배분합니다 (40개 미만이므로 Batch 1개). 배분 직후: 8개 모두 combo.used_at = datetime.now(tz=KST)로 갱신. test_used_combinations_not_redispatched()가 검증하듯, 같은 dispatch()를 다시 호출하면 used_at IS NULL인 조합이 0개이므로 새로운 배치가 생성되지 않습니다. 새 키워드를 추가한 후 ComboGenerator를 다시 실행하면 새 조합들만 used_at=NULL로 추가되어 다음 dispatch()에서 처리됩니다.

4.2 combination_keywords (association table)

■ Execution Zone

Column	Type	Constraint	Description
combination_id	BIGINT FK	PK, combinations.id, CASCADE	조합
keyword_id	INTEGER FK	PK, keywords.id, CASCADE	키워드

▶ WHY: 이 테이블이 나온 이유

하나의 Combination은 여러 Keyword를 참조하고 (region=강남, subject=수학), 하나의 Keyword는 여러 Combination에 속합니다 (강남은 수학, 영어, 과학 조합 모두에 등장). 이 다대다 관계를 해소하는 표준 association table입니다.

▶ WITHOUT: 이 테이블이 없다면

Combination에 keyword_ids를 JSON 배열로 저장해야 합니다. 특정 키워드가 포함된 조합을 조회하는 WHERE keyword_id = ? 쿼리가 불가능합니다. 키워드 삭제 시 참조 무결성을 보장할 수 없습니다.

► SOLVES: 이 테이블이 해결하는 문제

다대다 관계의 정규화와 참조 무결성. combo.keywords로 ORM에서 직접 접근 가능하며, CASCADE DELETE로 조합이 삭제되면 연결도 자동 삭제됩니다. job_builder._build_values(combo)가 combo.keywords를 순회하여 slug→value dict를 생성합니다.

► EXAMPLE: 실제 동작 예시

Combination(id=100)이 (강남구, 수학, 과외) 조합이면, combination_keywords 테이블에 3개 행이 생성됩니다: (combination_id=100, keyword_id=1), (combination_id=100, keyword_id=4), (combination_id=100, keyword_id=6). 동시에 Keyword(id=1, value='강남구')는 다른 조합에서도 참조됩니다: (combination_id=101, keyword_id=1) → (강남구, 영어, 과외), (combination_id=102, keyword_id=1) → (강남구, 수학, 학원). 이렇게 Keyword '강남구'는 여러 Combination에 속하고, Combination(id=100)은 여러 Keyword를 가집니다. test_each_combination_has_correct_keyword_count()가 검증하듯, 3-slot 캠페인의 모든 조합은 정확히 len(combo.keywords) == 3입니다. job_builder._build_values(combo)가 이 관계를 순회합니다: combo.keywords → sorted by category.id → {region: '강남구', subject: '수학', learning_type: '과외'}. 만약 이 관계를 JSON 배열 [1, 4, 6]으로 저장했다면, '강남구가 포함된 모든 조합'을 찾는 쿼리가 DB별로 완전히 다른 JSON 함수를 요구합니다.

4.3 batches

■ Execution Zone

Column	Type	Constraint	Description
id	INTEGER PK	AUTO	
campaign_id	INTEGER FK	campaigns.id	캠페인
account_id	INTEGER FK	accounts.id	실행 계정
scheduled_at	DATETIME	NOT NULL	예정 시각
status	VARCHAR(16)	default 'pending'	pending/running/completed/failed
worker_pid	INTEGER	NULLABLE	실행 중인 프로세스 ID
started_at	DATETIME	NULLABLE	실행 시작 시각
completed_at	DATETIME	NULLABLE	실행 완료 시각

► WHY: 이 테이블이 나온 이유

BatchDispatcher가 최대 40개의 Combination을 하나의 Account에 묶어 배분한 결과입니다. 하나의 Batch는 '어떤 계정으로, 언제, 어떤 조합들을 처리하라'는 작업 묶음입니다.

► WITHOUT: 이 테이블이 없다면

Combination을 개별적으로 실행하면, 같은 계정에 동시에 여러 작업이 겹칠 수 있습니다. 배치 단위 상태 관리(전체 성공/부분 실패)가 불가능합니다. worker_pid로 중복 실행을 감지하는 것도 불가능합니다.

► SOLVES: 이 테이블이 해결하는 문제

작업의 원자적 묶음과 상태 관리. BatchWorker.run_batch()가 batch.status를 running→completed/failed로 전이합니다. worker_pid로 이미 실행 중인 배치를 감지하고, scheduled_at으로 시간순 실행을 보장합니다.

► EXAMPLE: 실제 동작 예시

캠페인에 90개의 미사용 조합이 있고, 가용 계정이 3개(blog_a, blog_b, blog_c)일 때 BatchDispatcher.dispatch()가 호출됩니다: _chunk([combo_1...combo_90], 40) → [[combo_1...40], [combo_41...80], [combo_81...90]]. zip(accounts, chunks)로 매칭: Batch(account_id=blog_a, items=[combo_1...40], scheduled_at=10:00), Batch(account_id=blog_b, items=[combo_41...80], scheduled_at=10:10), Batch(account_id=blog_c,

items=[combo_81...90], scheduled_at=10:20). scheduled_at은 10분 간격(SCHEDULE_JITTER_MINUTES)으로 분산됩니다. test_batch_items_never_exceed_batch_size()가 검증하듯, 어떤 배치도 40개(BATCH_SIZE)를 초과하지 않습니다. 세 번째 배치는 10개만 가지며, 이것도 정상입니다. BatchWorker가 첫 번째 배치를 실행하면: batch.status = 'pending' → 'running' (worker_pid=현재 프로세스) → 'completed' (40개 중 1개라도 성공) 또는 'failed' (전부 실패).

4.4 batch_items

■ Execution Zone

Column	Type	Constraint	Description
id	BIGINT PK	AUTO	
batch_id	INTEGER FK	batches.id	소속 배치
combination_id	BIGINT FK	combinations.id	대상 조합
status	VARCHAR(16)	default 'pending'	pending/running/completed/failed
result_url	VARCHAR(512)	NULLABLE	발행된 포스트 URL
error_message	TEXT	NULLABLE	에러 메시지
completed_at	DATETIME	NULLABLE	완료 시각

► WHY: 이 테이블이 나온 이유

배치 내 개별 작업의 성공/실패를 추적합니다. 40개 중 38개 성공, 2개 실패 같은 세분화된 결과가 필요합니다. 실패한 item만 재시도하려면 item 단위 상태가 필수입니다.

► WITHOUT: 이 테이블이 없다면

배치가 성공이면 40개 모두 성공, 실패면 40개 모두 실패로 기록됩니다. 1개 실패 때문에 39개를 다시 실행해야 하는 낭비가 발생합니다. 어떤 조합이 실패했고, 왜 실패했는지 알 수 없습니다.

► SOLVES: 이 테이블이 해결하는 문제

개별 작업의 독립적 상태 관리. BatchWorker._process_item()이 각 item의 status와 error_message를 독립적으로 기록합니다. UNIQUE(batch_id, combination_id)로 같은 배치에 같은 조합이 중복 배분되지 않습니다. result_url로 발행 결과를 추적하고, error_message로 실패 원인을 진단합니다.

► EXAMPLE: 실제 동작 예시

Batch(id=7, account_id=blog_b)에 40개의 BatchItem이 있습니다. BatchWorker.run_batch(batch)가 실행되면 for item in batch.items 루프가 돌립니다: item_1: build_posting_spec(combo_41) → spec → runner.run(spec, editor) → 성공 → item.status='completed', item.completed_at=now(). item_2: ... 성공. item_15: runner.run()에서 Gemini API가 429(Rate Limit)를 반환 → except에서 캐치 → item.status='failed', item.error_message='RateLimitError: 429'. item_16~40: 계속 정상 실행 (item_15의 실패가 전파되지 않음). 최종 결과: success_count=39, failed=1 → batch.status='completed' (1개라도 성공이므로). BatchWorker.run_batch()는 [ItemResult(item_id=1, success=True), ..., ItemResult(item_id=15, success=False, error='RateLimitError: 429'), ...] 리스트를 반환합니다. 운영자는 status='failed'인 item만 조회하여 재시도할 수 있습니다: SELECT * FROM batch_items WHERE batch_id=7 AND status='failed'. 만약 40개 전부 실패하면 batch.status='failed'가 됩니다.

5. 전체 ERD Zone Map

21개 테이블을 네 개의 Zone으로 분류한 전체 지도입니다. 화살표는 Foreign Key 방향(자식 → 부모)을 나타냅니다.

Zone	테이블	핵심 FK 관계
Identity (보라)	users platforms accounts media	accounts → users, platforms media → users
Keyword (청록)	keyword_categories keywords affixes	keywords → keyword_categories, keywords(self) affixes → keywords
Campaign (주황)	campaigns campaign_slots spacing_rules campaign_keyword_picks campaign_palettes palette_items post_layouts layout_slots publish_presets run_presets	campaigns → users, platforms, post_layouts, publish_presets, run_presets campaign_slots → campaigns, keyword_categories campaign_keyword_picks → campaigns, categories, keywords campaign_palettes → campaigns palette_items → campaign_palettes layout_slots → post_layouts
Execution (파랑)	combinations combination_keywords batches batch_items	combinations → campaigns, spacing_rules combination_keywords → combinations, keywords batches → campaigns, accounts batch_items → batches, combinations

Table 5-1. 전체 ERD Zone Map

6. 테이블 간 핵심 질문 Q&A

Q: Campaign에 3개의 nullable FK(layout, publish_preset, run_preset)가 있는데, 이 세 설정을 campaigns 테이블의 JSON 컬럼 하나로 합치면 안 되나요?

JSON 하나로 합치면 레이아웃을 여러 캠페인이 공유할 수 없습니다. '강남 캠페인'과 '서초 캠페인'이 동일한 레이아웃을 쓸 때, FK 방식은 같은 layout_id를 참조하여 레이아웃 수정이 양쪽에 즉시 반영됩니다. JSON 방식은 두 곳에 복사되어 동기화 문제가 발생합니다.

Q: affixes 테이블 대신 keywords.display_value를 쓰면 안 되나요?

display_value는 키워드 하나에 하나의 표기만 제공합니다. 하지만 '강남구'에서 '구'를 때로는 떼고 때로는 유지해야 합니다. affixes의 active 플래그가 이 동적 전환을 가능하게 합니다. display_value는 UI 표시용이고, affixes는 런타임 파생 규칙입니다.

Q: combination_keywords 대신 combinations.config JSON에 keyword_ids를 저장하면?

JSON 배열로 저장하면 '강남'이 포함된 모든 조합'을 찾는 쿼리가 DB 엔진마다 다른 JSON 함수를 써야 합니다. association table은 표준 SQL JOIN으로 해결됩니다. 또한 CASCADE DELETE로 참조 무결성이 자동 관리됩니다.

Q: batches와 batch_items를 합쳐서 하나의 테이블로 만들면?

배치 수준 속성(account_id, scheduled_at, worker_pid)과 아이템 수준 속성(combination_id, result_url, error_message)의 관심사가 다릅니다. 합치면 40개 row가 같은 account_id, scheduled_at을 중복 저장합니다. 또한 배치 전체 상태(completed if any success)를 계산하는 로직이 복잡해집니다.

Q: campaign_palettes 대신 campaigns.config JSON 안에 salts를 넣으면?

동작은 하지만, salt 풀이 수백 개일 때 campaigns 행의 JSON이 비대해집니다. 개별 salt 값의 active 토글, 정렬 순서 변경이 전체 JSON을 다시 쓰는 작업이 됩니다. 별도 테이블은 palette_items 한 행만 UPDATE하면 됩니다.

Q: spacing_rules.pattern이 JSON인 이유는? 별도 테이블로 정규화하면?

패턴의 구조가 캠페인마다 완전히 다를 수 있어서 고정 스키마로 정의하기 어렵습니다. JSON은 유연한 구조를 허용하면서도, 규칙 자체가 소수(보통 1-5개)이므로 정규화의 이점(인덱싱, JOIN)보다 유연성의 이점이 큼니다.

이 해설서는 factory/models.py의 모든 21개 테이블을 다룹니다. 각 테이블의 존재 이유를 이해하면, 코드의 job_builder, combo_generator, batch_worker가 왜 그런 구조인지가 자연스럽게 따라옵니다.